Efficient Sequence Models: A Survey of Linear Attention, Test-Time Memory and KV Cache Compression

Amara Diallo, Felix Grunewald, Mei Tanaka, Lucas Moreau (Collegium for Efficient AI)

Abstract

Long-context language models are limited by the cost of attention and by the memory of the key-value cache. This survey reviews three responses to these limits: linear attention and other fixed-state recurrences, memory modules that learn while the model runs, and compression of the key-value cache by quantization and eviction. We organise more than forty methods into a common taxonomy, compare their reported results, and list open problems.

1 Introduction

A Transformer decoding a long input pays twice. Each new token attends to every previous token, and every previous token must be kept in the key-value cache. For long documents, code bases and agent transcripts the cache, rather than the weights, determines how many requests fit on an accelerator and how long each of them may be.

Work on this problem has been fragmented across communities. Architecture papers replace attention with recurrences that keep a fixed-size state, systems papers compress the cache of an unchanged model, and a newer line of work treats memory as a model that is trained at inference time. These directions are rarely compared with each other, and their benchmarks differ.

This survey brings them together. Section 2 introduces a taxonomy. Sections 3 to 5 review each family with a catalogue of representative methods, Section 6 discusses hybrid designs, Section 7 compares reported results and Section 8 lists open problems.

2 Taxonomy

We group efficient sequence models into three families according to what they change. The first family changes the attention operator itself so that the past is summarised in a state of fixed size. The second family adds a memory that is written and read by learning rather than by storage. The third family leaves the model unchanged and reduces the cache, either by storing each entry in fewer bits or by storing fewer entries.

The families trade different resources. Fixed-state recurrences save both computation and memory but lose precise recall. Learned memories recover recall at the cost of extra computation during inference. Cache compression keeps the original model's quality best but saves only memory, not the quadratic attention computation.

3 Linear Attention and Fixed-State Recurrences

Linear attention writes each key-value pair into a matrix-valued state and reads it with the query, so the cost of each step is constant. The early variants simply summed the outer products, and the state saturated once it held as many pairs as its key dimension. Two ideas made later models competitive: decay gates, which shrink the state at every step, and the delta rule, which overwrites the value stored for a key.

LUMEN [12] combines both ideas in a gated delta rule, in which a scalar gate multiplies the state before a delta step writes the new association. It trains with a chunk-wise parallel algorithm and reports a lower WikiText perplexity than a Transformer baseline of the same size. ORBIT [13] adds rotary position encoding to chunk-wise linear attention and keeps the training cost linear in the sequence length.

3.1 Catalogue of linear attention variants

The following methods are representative of the linear attention family. Each entry gives the main idea and the headline result reported by its authors; numbers are not directly comparable across entries because training data and model sizes differ.

Alder (2021) belongs to the linear attention family. It replaces the softmax with a positive random feature map so that attention can be computed as a running sum. It reports a WikiText perplexity of 21.4 at 350M parameters.

Within the linear attention line of work, Basalt (2022) is notable for one design choice. It adds a learned scalar decay to every head so that the state forgets at a rate chosen per head. It reports a WikiText perplexity of 19.8 at 350M parameters.

Cobalt, published in 2022, is a linear attention method. It normalises the read-out by the running sum of keys, which keeps the output scale stable over long inputs. Its authors measure a 1.9 times speedup over softmax attention at 8K tokens.

Drift (2023) belongs to the linear attention family. It shifts each token's key by one position before writing it, giving the state a cheap form of local order. It improves MQAR recall by 6 points over Alder.

Within the linear attention line of work, Ember (2023) is notable for one design choice. It writes keys and values into a state whose rows are sorted by a hash of the key, so similar keys share rows. It reports a LAMBADA accuracy of 45.0 at 760M parameters.

Fennel, published in 2024, is a linear attention method. It mixes a short convolution into the key projection so that n-gram statistics reach the state. It reaches a WikiText perplexity of 18.1 at 760M parameters.

Garnet (2024) belongs to the linear attention family. It uses data-dependent gates on both the rows and the columns of the state, at a cost of one extra projection. It reports a WikiText perplexity of 17.6 at 1.3B parameters.

Within the linear attention line of work, Jasper (2024) is notable for one design choice. It keeps two states with different decay rates and lets each head read a learned mixture of them. It improves MQAR recall by 9 points over Basalt.

Kestrel, published in 2024, is a linear attention method. It writes only every second token into the state and interpolates the reads in between, halving the update cost. It trains 1.6 times faster than Fennel at equal perplexity.

Lichen (2025) belongs to the linear attention family. It replaces the outer-product write with a low-rank update whose rank grows with the length of the input. It reports a WikiText perplexity of 17.1 at 1.3B parameters.

Within the linear attention line of work, Moss (2025) is notable for one design choice. It applies a learned orthogonal transition to the state before each write, so information rotates rather than decays. It keeps 80% MQAR recall at 512 key-value pairs.

4 Memory Modules that Learn at Inference

A fixed-size state must decide at write time what to keep. Learned memories instead fit a small network to the stream of keys and values while the model runs, so that capacity is spent where the input is hard to predict. This idea, known as test-time training, turns memorization into an optimisation problem solved during inference.

TIDE [14] is the most complete example. Its memory is a two-layer perceptron trained by gradient descent on a key-to-value reconstruction loss, with momentum and a gate that clears old content, and tokens that the memory predicts badly change it the most. Used as extra context for attention, it reports the best LongRecall accuracy among the methods compared here.

4.1 Catalogue of memory modules

The following methods add an explicit or learned memory to a sequence model. They differ in what is stored, when it is written, and whether the memory is trained in advance or at inference.

Heron (2020) belongs to the recurrent memory family. It keeps an external memory of past segment summaries that later segments read with cross-attention. It improves LongRecall accuracy at 64K tokens to 35.0%.

Within the recurrent memory line of work, Iris (2021) is notable for one design choice. It compresses each segment into eight memory slots with a learned pooling layer before discarding the tokens. It reports 28.4% LongRecall accuracy at 64K tokens.

Juniper, published in 2022, is a recurrent memory method. It trains a fast-weight matrix with a Hebbian rule inside each layer and resets it at document boundaries. It is 1.2 times slower than a Transformer at 4K tokens.

Kelp (2023) belongs to the recurrent memory family. It stores key-value pairs in a differentiable hash table that grows by doubling when it is three quarters full. It keeps 90% recall up to 128K tokens on its own benchmark.

Within the recurrent memory line of work, Larch (2023) is notable for one design choice. It updates a small recurrent network by one gradient step per segment, an early form of test-time training. It reports 41.7% LongRecall accuracy at 64K tokens.

Marrow, published in 2024, is a recurrent memory method. It mixes a sliding-window attention with a recurrent summary through a learned scalar gate per channel. It reaches a WikiText perplexity of 18.9 at 760M parameters.

Nimbus (2024) belongs to the recurrent memory family. It routes each token either to a memory write or to a skip path, chosen by a learned router. It halves the number of memory writes with no loss in recall.

Within the recurrent memory line of work, Nettle (2024) is notable for one design choice. It keeps a sparse set of memory slots addressed by learned keys and writes each token into its two nearest slots. It reports 44.9% LongRecall accuracy at 64K tokens.

Osprey, published in 2025, is a recurrent memory method. It trains its memory with a second-order optimiser at inference, using a diagonal estimate of the curvature. It reports 63.0% LongRecall accuracy at 64K tokens.

Plover (2025) belongs to the recurrent memory family. It stores a compressed copy of every thousandth token alongside a recurrent summary of the tokens in between. It keeps 85% recall up to one million tokens on its own benchmark.

Within the recurrent memory line of work, Quartz (2025) is notable for one design choice. It shares one memory across the layers of a model and lets each layer write to its own region of it. It uses 30% fewer parameters than a per-layer memory of equal recall.

5 Compressing the Key-Value Cache

Cache compression leaves the trained model untouched and reduces the memory of its key-value cache. Quantization keeps every cached token but stores each number in fewer bits. Eviction keeps full precision but stores only a budget of tokens, chosen by recency, position or the attention they receive.

Among quantization methods, SQUEEZE [15] stores the cache in two bits, quantizing keys per channel because of their fixed outlier channels and values per token, and reports a memory reduction of more than five times. Among eviction methods, HARBOR [16] keeps the first tokens, a recent window and the tokens that received the most attention, and keeps nearly all of the full-cache accuracy with a fifth of the cache. The two are complementary, and HARBOR's authors report a larger reduction when they are applied together.

5.1 Catalogue of cache quantization methods

The following methods reduce the number of bits per cached number. Most of them are training-free and differ in how they group numbers for a shared scale and how they treat outliers.

Opal (2022) belongs to the cache quantization family. It stores the cache in eight-bit integers with one scale per attention head. It halves cache memory with no measurable accuracy loss.

Within the cache quantization line of work, Pumice (2023) is notable for one design choice. It quantizes keys and values per token to four bits and keeps the attention sink tokens in full precision. It reduces cache memory by 3.6 times.

Quill, published in 2023, is a cache quantization method. It applies a random rotation to keys before quantizing them, which spreads outliers across channels. It loses 1.1 points on LongSuite at three bits.

Ripple (2024) belongs to the cache quantization family. It quantizes the cache progressively, lowering the precision of tokens as they age from eight bits to two. It reduces cache memory by 4.4 times with a loss of 0.9 points on LongSuite.

Within the cache quantization line of work, Sorrel (2024) is notable for one design choice. It shares one set of quantized keys across adjacent layers whose keys are highly correlated. It reduces cache memory by 6.0 times.

Tamarind, published in 2024, is a cache quantization method. It learns a vector codebook per layer and stores each cached key as the index of its nearest code. It reaches 1.5 bits per number with a loss of 2.3 points.

Umber (2025) belongs to the cache quantization family. It mixes precisions per head, giving more bits to heads whose attention is spread over many tokens. It loses 0.6 points on LongSuite at an average of 2.5 bits.

Within the cache quantization line of work, Velvet (2025) is notable for one design choice. It quantizes values with a per-token scale and stores the scales themselves in eight bits. It reduces cache memory by 4.8 times.

Willow, published in 2025, is a cache quantization method. It keeps keys in four bits and values in two, after finding that attention scores are more sensitive to key error. It loses 0.5 points on LongSuite.

Yew (2025) belongs to the cache quantization family. It compresses the cache along the head dimension with a learned projection before quantizing it. It reduces cache memory by 7.1 times with a loss of 1.8 points.

Within the cache quantization line of work, Zinnia (2025) is notable for one design choice. It decides the precision of each block of tokens from the norm of its keys, spending bits on the largest keys. It reaches an average of 2.2 bits per number.

5.2 Catalogue of cache eviction methods

The following methods keep a subset of the cached tokens. They differ in the score that ranks tokens and in how the budget is spread across layers and heads.

Vetch (2022) belongs to the cache eviction family. It evicts tokens uniformly at random once the cache is full, a baseline that is surprisingly hard to beat on short tasks. It keeps 81% of full-cache accuracy at a 50% budget.

Within the cache eviction line of work, Wren (2023) is notable for one design choice. It keeps a fixed window of recent tokens plus the first few tokens of the sequence. It keeps 72% of full-cache accuracy at a 20% budget.

Xylem, published in 2023, is a cache eviction method. It merges evicted tokens into their most similar kept neighbour instead of dropping them. It keeps 93% of full-cache accuracy at a 20% budget.

Yarrow (2024) belongs to the cache eviction family. It chooses the budget per layer from the entropy of the layer's attention, giving more cache to diffuse layers. It keeps 95% of full-cache accuracy at a 15% budget.

Within the cache eviction line of work, Zephyr (2024) is notable for one design choice. It scores tokens with an observation window of the last 32 queries before generation starts. It keeps 96% of full-cache accuracy at a 10% budget on retrieval tasks.

Aster, published in 2025, is a cache eviction method. It offloads evicted entries to CPU memory and fetches them back when a query attends strongly to their page. It recovers 99% of full-cache accuracy at a 10% GPU budget.

Bracken (2025) belongs to the cache eviction family. It predicts future attention with a small learned probe and evicts the tokens the probe expects to be ignored. It keeps 97% of full-cache accuracy at a 15% budget.

Within the cache eviction line of work, Cedar (2025) is notable for one design choice. It evicts whole pages of sixteen tokens at a time, which keeps the cache layout friendly to paged attention kernels. It keeps 94% of full-cache accuracy at a 20% budget.

Dahlia, published in 2025, is a cache eviction method. It evicts different tokens in different heads, so that each head keeps the tokens it attends to most. It keeps 96% of full-cache accuracy at a 15% budget.

Elm (2025) belongs to the cache eviction family. It shrinks the budget of deeper layers, where attention concentrates on fewer tokens. It keeps 95% of full-cache accuracy at an average budget of 12%.

Within the cache eviction line of work, Fern (2025) is notable for one design choice. It re-scores the cache only every 64 decoding steps, which removes most of the cost of tracking attention. It decodes 1.3 times faster than per-step scoring at equal accuracy.

6 Hybrid Designs

Several recent models combine families: a few softmax attention layers restore precise recall in an otherwise linear model, or a recurrent summary extends a sliding-window attention. Hybrids currently give the best quality per unit of memory, at the cost of more complex kernels and serving systems.

6.1 Catalogue of hybrid models

The following models mix attention with recurrent or learned components in one network.

Cress (2023) belongs to the hybrid family. It interleaves one softmax attention layer after every six linear attention layers. It matches Transformer recall on MQAR at 1.3B parameters.

Within the hybrid line of work, Dune (2024) is notable for one design choice. It runs a state space layer and a sliding-window attention in parallel in every block and adds their outputs. It reaches a WikiText perplexity of 16.9 at 1.3B parameters.

Egret, published in 2024, is a hybrid method. It uses global attention only in the middle third of the layers and linear attention elsewhere. It reduces cache memory by 3.0 times relative to a Transformer.

Flint (2024) belongs to the hybrid family. It shares a single key-value cache across all of its softmax layers, which are spaced eight layers apart. It reduces cache memory by 8.0 times.

Within the hybrid line of work, Gorse (2025) is notable for one design choice. It replaces the feed-forward block of every fourth layer with a Kolmogorov-Arnold layer. It reports a WikiText perplexity of 17.2 at 760M parameters.

Hazel, published in 2025, is a hybrid method. It distils a pretrained Transformer into a linear attention model by matching attention maps layer by layer. It recovers 96% of the teacher's accuracy with 2B training tokens.

Ibex (2025) belongs to the hybrid family. It decides per token whether to attend with softmax or with the recurrent state, using a learned threshold. It uses softmax attention for 12% of tokens on average.

Within the hybrid line of work, Jacaranda (2025) is notable for one design choice. It runs a test-time trained memory in parallel with a linear attention layer and gates between them per token. It reports 66.4% LongRecall accuracy at 64K tokens.

Kumquat, published in 2025, is a hybrid method. It stores the key-value cache of its few softmax layers in two bits while its linear layers keep a state. It reduces cache memory by 21 times relative to a Transformer.

Lotus (2025) belongs to the hybrid family. It places a sliding-window attention of 2K tokens in every layer and a global memory in every eighth layer. It reaches a WikiText perplexity of 16.6 at 1.3B parameters.

Within the hybrid line of work, Mallow (2025) is notable for one design choice. It converts a pretrained hybrid into a pure recurrent model for deployment by distilling its attention layers. It keeps 94% of the hybrid's accuracy.

7 Comparison of Reported Results

Reported results are hard to compare: papers train on different data, at different sizes, and evaluate on different benchmarks. Where a common benchmark exists the picture is consistent. On language modelling, gated recurrences now match Transformers of the same size. On recall at long contexts, learned memories and hybrids lead, while pure fixed-state models fall behind beyond 16K tokens. Cache compression methods lose less than one point of accuracy down to about two bits or a 20% budget, and degrade quickly below that.

Table 4: The methods discussed in Sections 3 to 5 and the benchmarks they report.

Method	Family	Benchmark	Section
LUMEN	linear	WikiText ppl	3
TIDE	memory	LongRecall	4
SQUEEZE	quantization	LongSuite	5
HARBOR	eviction	LongSuite	5

8 Open Problems

We see four open problems. First, there is no shared long-context benchmark on which architectures and cache compression methods are evaluated together. Second, the interaction between compression and learned memory is unexplored: a model with test-time memory may need a smaller cache, but no work measures this. Third, eviction errors cannot be undone, and methods that can recover evicted entries cheaply are only beginning to appear. Fourth, efficiency is reported in incomparable units, and we recommend reporting memory per token and tokens per second at a fixed context length.

9 Conclusion

Efficient sequence modelling has moved from approximating attention to redesigning memory. The most promising systems combine a compact recurrent state, a small amount of exact attention and a compressed cache, and the field needs common benchmarks to tell which combination is best.

References

[1] K. Marr, S. Oduya and T. Brandt. Kernelized attention with feature maps. In Proceedings of the Learning Systems Conference, 2020.

[2] R. Castellan. The delta rule revisited for linear transformers. arXiv preprint, 2021.

[3] A. Petrov and L. Huang. Decay gates for recurrent attention. In Proceedings of the Sequence Modelling Workshop, 2023.

[4] D. Farrow and N. Iyer. Test-time training layers for sequence models. arXiv preprint, 2024.

[5] T. Okafor and R. Lind. Where attention parks its mass in long sequences. In Proceedings of the Sequence Modelling Workshop, 2023.

[6] G. Alvarez and M. Stone. Outlier channels in transformer activations. Transactions on Efficient Computing, 2022.

[12] M. Okonkwo, T. Vaidya and E. Lindqvist. LUMEN: Linear attention with a gated delta rule. In Proceedings of the Learning Systems Conference, 2024.

[13] B. Achterberg and S. Moreno. ORBIT: Rotary position encoding for chunk-wise linear attention. arXiv preprint, 2024.

[14] S. Brandauer, K. Mensah and L. Qiaoyu. TIDE: Learning to memorize at test time with surprise-gated updates. arXiv preprint, 2025.

[15] P. Raman, O. Weiss and D. Feld. SQUEEZE: Two-bit key-value cache quantization with per-channel keys. arXiv preprint, 2024.

[16] N. Ferreira and J. Ekholm. HARBOR: Heavy-hitter aware eviction for long-context KV caches. arXiv preprint, 2024.